

Propuesta de arquitectura en AWS

Prueba Técnica Backend Developer - API de Solicitudes Institucionales

1. Resumen de servicios seleccionados:

Servicio	Función
VPC	Red privada aislada; separa componentes públicos y privados.
Route 53	Resuelve el dominio hacia la IP del ALB (DNS administrado).
ALB (Application Load Balancer)	Punto de entrada único desde internet, maneja HTTPS y enruta hacia los contenedores.
AWS WAF	Filtra tráfico malicioso (SQL injection, XSS, bots) antes de llegar al ALB.
ECS Fargate	Ejecuta los contenedores (API y consumidor) sin administrar servidores.
ECR	Almacena las imágenes Docker construidas en el pipeline de CI/CD.
RDS (PostgreSQL)	Base de datos administrada, en subnet privada.
Secrets Manager	Almacena credenciales de BD y claves de firma de JWT.
CloudWatch	Centraliza logs, métricas y alarmas.
IAM	Roles y permisos de mínimo privilegio para cada servicio.

2. Contenedores e imágenes

Las imágenes de backend y consumer se construyen en un pipeline de CI/CD y se publican en **ECR**. **ECS Fargate** ejecuta cada una como un task independiente, definiendo CPU/memoria por servicio. Fargate elimina la necesidad de administrar instancias EC2: AWS gestiona el hardware subyacente.

3. Punto de entrada: ALB

Se selecciona **ALB** sobre API Gateway porque el sistema atiende un frontend propio y servicios internos controlados, no terceros externos con necesidad de API Keys o planes de uso diferenciados. El ALB ofrece menor latencia y costo por request; autenticación (JWT) y rate limiting se implementan en la propia aplicación, dando mayor control sobre las reglas de negocio. Si el sistema se expusiera a integraciones externas, API Gateway sería la migración natural.

Configuración:

- **Listener** en el puerto 443 (HTTPS), con certificado gestionado por AWS Certificate Manager. Un listener adicional en el puerto 80 redirige a 443.
- **Listener rule** única: todo el tráfico se enruta al Target Group de la API.
- **Health check** del Target Group apunta a GET /health, con intervalo de 30s.

4. Red: segmentación pública/privada

Internet -> Route 53 (DNS) -> WAF -> ALB (subnet pública) -> ECS Fargate (subnet privada) -> RDS (subnet privada). Ni la API ni PostgreSQL tienen IP pública ni ruta directa a internet. Solo el ALB vive en la subnet pública.

5. Reglas de acceso (Security Groups)

Un Security Group (SG) es un firewall a nivel de recurso en AWS: define qué tráfico puede entrar o salir de un componente específico (ALB, ECS, RDS), y puede restringirse no solo por IP, sino por el propio Security Group de origen; por ejemplo, "aceptar tráfico únicamente si proviene de recursos con el SG del ALB". Esto garantiza que cada salto de la cadena (ALB - ECS - RDS) solo confíe en el componente inmediatamente anterior, incluso si en el futuro se agregan nuevos recursos a la misma VPC.

Origen	Destino	Puerto	Regla
Internet	ALB	443	Permitido
ALB	ECS (API)	8000	Solo desde SG del ALB

ECS (API)	RDS	5432	Solo desde SG de ECS
Cualquier otro origen	RDS	5432	Denegado

6. Autenticación y autorización

- **Usuarios (frontend - API):** JWT firmado por la API tras login. El frontend envía Authorization: Bearer <token> en cada request; la API valida firma y expiración antes de procesar. La clave de firma se guarda en Secrets Manager.
- **Servicio a servicio** (ej. consumer - API): el Task de ECS del consumidor usa un IAM Role propio; la comunicación interna se autentica con un token de servicio almacenado en Secrets Manager, distinto del JWT de usuario.

7. CORS, rate limiting y protección de tráfico

- **CORS:** lista blanca de dominios permitidos (el dominio del frontend), configurada en la aplicación FastAPI.
- **Rate limiting:** reglas en AWS WAF (por IP) y, opcionalmente, límites adicionales por usuario/token dentro de la aplicación.
- **WAF:** reglas administradas de AWS (OWASP Top 10) delante del ALB, bloqueando patrones de ataque conocidos antes de que lleguen a la aplicación.

8. Gestión de secretos, logs y trazabilidad

- **Secretos:** contraseña de RDS y clave de JWT en Secrets Manager; los Tasks de ECS los obtienen en tiempo de ejecución vía IAM Role, nunca escritos en la imagen ni en variables de entorno planas.
- **Logs:** cada Task envía sus logs (ya estructurados en JSON, con request_id) a CloudWatch Logs.
- **Métricas y alarmas:** CloudWatch Alarms sobre tasa de errores 5xx, latencia y uso de CPU/memoria de los Tasks.
- **Trazabilidad:** el request_id generado por el middleware permite correlacionar logs de una misma petición a través de los distintos componentes.

9. Escalabilidad, despliegue y reversión

- **Escalabilidad:** ECS Service Auto Scaling ajusta el número de Tasks según CPU/memoria o número de requests en el Target Group.
- **Despliegue:** rolling deployment nativo de ECS - levanta Tasks con la nueva versión, espera que pasen el health check, y retira las anteriores gradualmente, sin downtime.
- **Reversión:** ECS conserva las definiciones de Task anteriores (*Task Definitions* versionadas); revertir es re-desplegar la versión previa en segundos.
- **Nuevos servicios:** la arquitectura permite agregar nuevos Target Groups y Listener Rules sin modificar los servicios existentes.

Nota: esta es una propuesta inicial, con foco en cumplir los requisitos de seguridad de red, gestión de secretos y trazabilidad. Con experiencia adicional en AWS se podrá mejorar la propuesta de despliegue.